安全科学与工程学院

2024－2025 学年第一学期

《互联网应用服务开发与数据库实践》

题　目：校园兼职招聘管理系统
班　级：24034202
学　号：241240048
姓　名：黄潇萱
电子签（必填）：黄潇萱
教　师：邢艳
成　绩：

2025 年 1 月 10 日

系统设计文档

一．系统需求分析

1.1 系统开发背景

随着高校在校生规模持续扩大，大学生利用课余时间参加兼职工作已成为一种普遍现象。有关调查显示，超过六成的在校大学生有过兼职经历，兼职收入已成为部分学生重要的生活来源。然而，当前校园兼职的信息流转仍以 QQ 群、微信群、校园公告栏等分散渠道为主，存在以下突出问题：（1）信息来源分散，学生需要在多个渠道之间反复筛选，获取岗位信息的成本较高；（2）信息真实性缺乏保障，虚假招聘与兼职诈骗时有发生，学生权益容易受损；（3）供需匹配效率低，商家难以快速找到符合要求的在校学生，学生也难以快速找到与自身时间、地点相符的岗位；（4）申请、录用与评价环节缺少规范化管理，录用结果依赖线下通知，兼职结束后的反馈渠道缺失，双方权益均缺少有效保障。

基于上述背景，本课程设计开发一套面向校内学生与商家的校园兼职招聘管理系统。系统采用 B/S 架构，将岗位发布、信息检索、在线申请、录用管理和双向评价整合到同一平台，以规范化的线上流程替代分散的人工传递，提高校园兼职信息的可达性、真实性与管理效率。

1.2 系统目标

本系统的总体目标是构建一个功能完整、操作简便、运行稳定的校园兼职招聘管理平台，具体目标如下：

（1）为商家提供岗位在线发布、修改与下架功能，降低岗位信息的发布与维护成本；

（2）为学生提供多条件岗位检索、在线申请与申请状态跟踪功能，提高岗位获取效率；

（3）为管理员提供用户审核、岗位审核与评价审核功能，保障平台信息的真实性与规范性；

（4）通过视图、存储过程和触发器实现数据完整性与业务逻辑的自动化，保证录用过程的数据一致性；

（5）界面美观友好，系统具备良好的可维护性与可扩展性，便于后续功能扩展。

系统面向学生、商家、管理员三类用户，采用 B/S（Browser/Server）架构，用户通过浏览器即可访问，无需安装专用客户端。

1.3 功能需求

系统需求描述：系统按照用户角色将功能划分为学生端、商家端与管理员端三个功能域。学生端覆盖从岗位发现到兼职结束评价的完整链路，即浏览岗位、按条件检索、查看岗位详情、提交申请、跟踪申请状态、对已完成的兼职进行评分与评价；商家端覆盖岗位的发布与维护、申请者信息的查看以及录用与拒绝的决策；管理员端覆盖用户资质审核、岗位内容审核、评价内容审核与平台数据统计。三个功能域共享同一套数据模型，通过角色权限相互隔离。

具体功能如下：

1、用户注册与登录。学生与商家通过注册页面提交基本信息完成注册，管理员账号由系统预置；密码采用 BCrypt 加盐哈希方式存储，登录时按角色分流至不同界面。
2、岗位信息发布。商家填写岗位名称、岗位类型、薪资及计薪单位、工作地点、工作时段、招聘人数、报名截止日期与岗位描述后提交发布，新发布岗位默认为待审核状态。
3、岗位审核管理。管理员对待审核岗位进行审核，审核通过后岗位方可在前台展示并接受申请，审核驳回的岗位不对外展示。
4、岗位检索浏览。学生可按岗位类型、薪资范围、发布时间等条件组合检索，检索结果分页展示，点击可查看岗位详情与商家联系方式。
5、在线申请管理。学生对在招岗位提交申请，系统校验岗位状态、报名截止日期与重复申请；商家在申请列表中执行录用或拒绝操作，处理结果实时反馈给学生。
6、用户信息管理。管理员查看学生与商家的注册信息，对不符合要求的账号执行审核或禁用操作。
7、评价管理。学生对已录用的兼职进行 1～5 星评分并填写文字评价，评价提交后默认为待审核，管理员审核通过后在岗位详情页公开展示。
8、数据统计。管理员可查看注册用户数、在招岗位数与申请记录数等平台运行指标。

1.4 非功能需求

（1）性能需求。

①响应时间：常规网络环境下，登录、检索、申请等核心操作的页面响应时间不超过 3 秒。

②并发能力：系统应支持不少于 500 名用户同时在线，并在此负载下支持不少于 100 个并发请求。

上述指标的依据如下。以岗位检索页为例，该页面资源总量约 800KB（其中 HTML 50KB、CSS 80KB、JavaScript 120KB、图片与图标 550KB），校园网带宽按 10Mbps（约 1.25MB/s）计算，传输耗时约 0.64 秒；服务器端数据库查询在建立索引并使用存储过程预编译的条件下耗时为 0.2～0.5 秒；浏览器渲染耗时约 0.3 秒。三项相加，页面整体响应时间约为 1.1～1.5 秒，满足 3 秒以内的指标要求，也与 Nielsen 关于 3 秒为用户可接受上限的经验阈值相符。

并发指标按注册学生 10000 人、日均在线率 20%测算，日常在线用户约为 2000 人；参照校园招聘类系统的行为特征，同一时刻处于操作状态的用户约占在线用户的 25%，即 500 名并发用户；再按每位用户平均 5 秒发起一次请求、单次请求处理耗时 0.5 秒计算，峰值请求速率约为 50 个/秒。IIS 线程池与 SQL Server 连接池（默认最大 100 个连接）的容量均高于该峰值，故上述指标具备可行性。

（2）安全性需求。

①密码安全：用户密码采用 BCrypt 强哈希算法存储，每次注册随机生成 22 位盐值，成本因子统一取 10，数据库中不保存任何明文密码，可有效抵御彩虹表与暴力破解。

②访问控制：采用基于角色的访问控制（RBAC），学生、商家、管理员三种角色仅能访问各自被授权的页面与功能，未授权访问由页面基类在 Page_Load 阶段统一拦截。

③输入与输出安全：数据访问层统一使用参数化查询防止 SQL 注入；所有用户提交的内容在页面输出时经 HTML 编码处理，防止跨站脚本攻击（XSS）。
④文件上传安全：对宣传图等上传文件校验扩展名白名单（.jpg、.png）、文件大小上限 2MB 及内容类型，拒绝可执行文件上传。
⑤数据备份：数据库定期执行完整备份，备份文件与数据库文件分盘存放，以降低单点故障导致的数据丢失风险。

（3）可用性需求。

①界面规范：为量化“界面友好”的要求，制定统一的 UI 设计规范。配色方面，主色为#2C6CB0，辅助色为#E8F0FB，页面底色为#F5F7FA，警示色为#E74C3C，成功色为#27AE60，正文文字为#333333，次要文字为#666666，同一页面主色不超过 2 种；字号方面，页面标题 18px 加粗、区块标题 16px 加粗、正文 14px、辅助文字 12px，行高 1.5 倍；布局方面，内容区宽度 1200px 并水平居中，栅格间距 20px，表单标签与输入框垂直间距 12px，按钮高 40px、圆角 4px，导航栏高 56px 并固定置顶；交互方面，控件悬停时颜色加深 10%，提交前由 JavaScript 完成校验并给出明确提示，操作结果均有文字反馈；一致性方面，全站统一引用 style.css，导航栏、页脚与控件样式保持一致。
②稳定性：系统正常运行时间不低于 99%，对异常输入具备容错能力，出错时给出可理解的提示而不暴露数据库错误细节。

（4）兼容性需求。

①浏览器：支持 Chrome 90 及以上、Firefox 88 及以上、Edge 90 及以上版本。
②设备：支持 1366×768 及以上分辨率的桌面显示设备，在主流分辨率下页面布局不发生错位。

1.5 非功能需求验收标准

为保证非功能需求可验证，各项需求均给出对应的验收标准与验证方式，如表 1 所示。

表 1 非功能需求验收标准表（acceptance_criteria）

类别	指标	验收标准	验证方式
性能	页面响应时间	登录、检索、申请等核心操作响应时间不超过 3 秒	浏览器开发者工具 Network 面板实测 10 次取平均值
性能	并发能力	500 用户在线、100 并发请求下系统正常运行	使用 JMeter 模拟并发，错误率低于 1%
安全	密码存储	password 字段全部为 BCrypt 哈希值，无明文	抽查 student 与 employer 表数据比对
安全	访问控制	未授权角色访问他人页面时被拦截并跳转登录页	使用三类角色账号交叉访问测试
安全	注入与 XSS 防护	输入含 SQL 关键字与脚本代码时被正确转义，页面不报错	构造注入字符串与 script 脚本提交测试
可用性	界面规范	各页面配色、字号、布局符合 UI 设计规范	按 5 组规范逐页检查
可用性	稳定性	连续 30 天运行，累计故障时间少于 7.2 小时	统计服务器运行日志
兼容性	浏览器与设备	Chrome、Firefox、Edge 下功能与样式保持一致	三种浏览器逐一验收

二．系统总体设计

系统采用 B/S 三层架构，自下而上划分为数据访问层（DAL）、业务逻辑层（BLL）和表现层（UI）。数据访问层基于 ADO.NET 实现，负责与 SQL Server 交互并统一使用参数化命令；业务逻辑层以 C#业务类封装岗位发布、申请处理、审核等业务规则；表现层由 ASP.NET Web 窗体页面构成，负责界面展示与用户交互。三层之间通过明确的接口调用，任一层内部实现的变化不会影响其他层，从而提升系统的可维护性、可扩展性与安全性。

各功能模块说明如下。系统功能模块划分如图 1 所示。

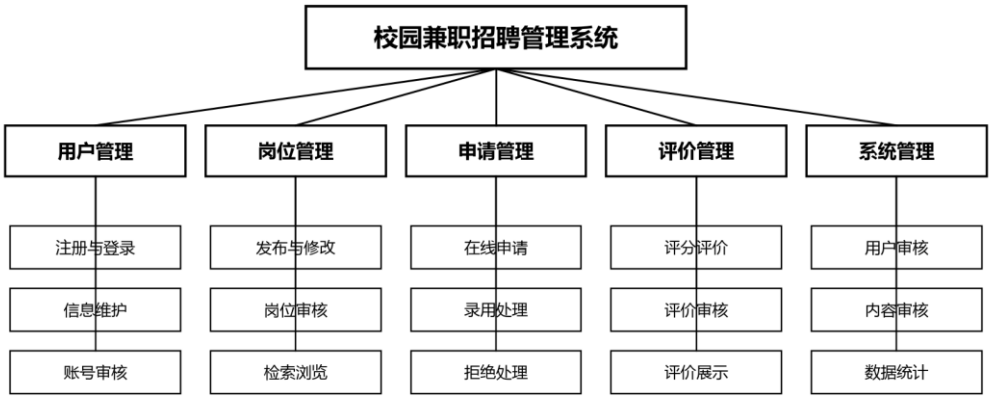


图 1 校园兼职招聘管理系统功能模块图

2.1 用户管理模块设计

用户管理模块面向学生、商家与管理员三类角色。学生注册时需填写学号、姓名、性别、学院、专业、年级、联系电话与邮箱，其中学号为唯一标识；商家注册时需填写商家名称、联系人、联系电话、地址与商家简介；管理员账号由系统在部署阶段预置。所有密码在业务逻辑层经 BCrypt 算法加盐哈希后写入数据库，成本因子统一取 10，登录时调用 BCrypt.Verify 方法比对，数据库中不保存明文。注册信息提交后进入待审核状态，管理员审核通过后账号方可正常登录，违规账号可被禁用。本模块涉及 student、employer、admin 三张表。

2.2 岗位管理模块设计

岗位管理模块是系统的核心模块，面向商家、学生与管理员三类角色。商家填写岗位名称、岗位类型、薪资与计薪单位、工作地点、工作时段、招聘人数、报名截止日期与岗位描述后提交发布，新岗位的审核状态默认为待审核；岗位信息被修改后审核状态自动重置为待审核，需重新审核。管理员负责审核岗位，审核通过的岗位状态置为在招并对外展示，驳回的岗位不展示。学生通过检索页按岗位类型、最低薪资与关键词组合查询，结果通过视图 v_job_detail 关联商家信息后分页展示，并按发布日期倒序排列。

商家可对已招满或已结束的岗位执行下架，下架采用状态更新的逻辑删除方式，保留历史数据以便统计与追溯。本模块主要涉及 job 表与视图 v_job_detail。

2.3 申请管理模块设计

申请管理模块面向学生与商家。学生在岗位详情页点击“申请该岗位”后，系统调用存储过程 proc_apply_job，该过程依次校验岗位是否存在且已通过审核、岗位是否处于在招状态、是否已过报名截止日期以及该学生是否已申请过该岗位，全部校验通过后在事务中写入申请记录，申请状态置为待处理。提交申请只产生待处理的申请记录，不改变岗位的已录用人数。

商家在申请管理页查看本企业岗位的申请列表，执行录用时调用存储过程 proc_hire_applicant，该过程在事务中对岗位记录加更新锁，确认已录用人数未达到招聘人数后，才将申请状态置为已录用并把已录用人数加一，从而保证并发场景下不会出现超额录用；执行拒绝时调用存储过程 proc_reject_applicant，仅将申请状态置为已拒绝。岗位满员后由触发器 trg_job_after_apply 自动将岗位状态更新为已招满。本模块主要涉及 application 表与 job 表。

2.4 评价管理模块设计

评价管理模块面向学生与管理员。当申请状态为已录用且兼职工作结束后，学生可对该次兼职进行 1～5 星评分并填写文字评价，评价记录同时写入岗位编号与商家编号，使评价既可用于反映岗位质量，也可用于反映商家信誉。评价提交后审核状态默认为待审核，管理员在后台完成审核，审核通过的评价在岗位详情页公开展示，用于辅助其他学生决策。本模块主要涉及 comment 表。

2.5 系统管理模块设计

系统管理模块面向管理员，承担平台的运营管理职能，包括用户审核与禁用、岗位审核、评价审核以及数据统计四项功能。用户审核用于确认学生与商家注册信息的真实性；岗位审核用于拦截虚假或违规岗位；评价审核用于过滤不当言论；数据统计通过聚合查询给出注册用户数、在招岗位数与申请记录数，为平台运营提供依据。本模块涉及全部六张数据表的查询与更新操作，并受管理员角色的权限控制。

三．数据库设计

本章按照概念结构设计、逻辑结构设计、物理结构设计与数据字典四个步骤完成数据库设计，目标数据库管理系统为 Microsoft SQL Server 2019，数据库名称为 CampusPartTime。

1．概念结构设计

概念结构设计采用 E-R 模型，通过对系统需求的抽象，确定系统中的实体、属性以及实体之间的联系。系统共包含六个实体，各实体的属性如下：

学生（学生编号，学号，姓名，性别，出生日期，学院，专业，年级，联系电话，邮箱，密码，注册日期，状态）

商家（商家编号，商家名称，联系人，联系电话，地址，商家简介，邮箱，密码，注册日期，状态）

管理员（管理员编号，管理员姓名，密码）

兼职岗位（岗位编号，商家编号，岗位名称，岗位类型，薪资，计薪单位，工作地点，工作时段，招聘人数，已录用人数，岗位描述，发布日期，报名截止日期，审核状态，岗位状态）

申请记录（申请编号，学生编号，岗位编号，申请日期，申请状态，备注）

评价（评价编号，学生编号，岗位编号，商家编号，评分，评价内容，评价日期，审核状态）
实体之间的联系如下：
（1）一个商家可以发布多个兼职岗位，商家与兼职岗位之间为 1:N 的“发布”联系；
（2）一个学生可以提出多条申请记录，学生与申请记录之间为 1:N 的“提出”联系；
（3）一个岗位可以接收多条申请记录，兼职岗位与申请记录之间为 1:N 的“接收”联系；
（4）一个学生可以发表多条评价，学生与评价之间为 1:N 的“发表”联系；
（5）一个岗位可以获得多条评价，兼职岗位与评价之间为 1:N 的“获得”联系；
（6）一个商家可以获得多条评价，商家与评价之间为 1:N 的“针对”联系；
（7）管理员对岗位进行审核，管理员与兼职岗位之间为 1:N 的“审核”联系。
根据上述实体、属性与联系，绘制系统 E-R 图如图 2 所示。

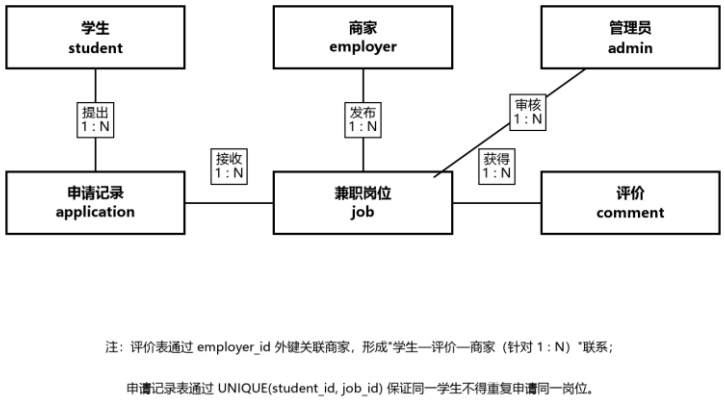


图 2 校园兼职招聘管理系统 E-R 图

2．逻辑结构设计

将 E-R 图按实体与联系转换为关系模式。其中，1:N 联系不单独建表，而是将“1”端实体的主键作为外键并入“N”端实体；评价与商家之间的“针对”联系同样以商家编号作为外键并入评价表。经过转化后可得如下关系（下划线表示主键，*表示外键）：
学生（学生编号，学号，姓名，性别，出生日期，学院，专业，年级，联系电话，邮箱，密码，注册日期，状态）
商家（商家编号，商家名称，联系人，联系电话，地址，商家简介，邮箱，密码，注册日期，状态）
管理员（管理员编号，管理员姓名，密码）
兼职岗位（岗位编号，商家编号*，岗位名称，岗位类型，薪资，计薪单位，工作地点，工作时段，招聘人数，已录用人数，岗位描述，发布日期，报名截止日期，审核状态，岗位状态）
申请记录（申请编号，学生编号*，岗位编号*，申请日期，申请状态，备注）
评价（评价编号，学生编号*，岗位编号*，商家编号*，评分，评价内容，评价日期，审核状态）

上述关系模式中，各非主属性均完全函数依赖于主键，且不存在传递函数依赖。例如学生表中的“学院”“专业”由学生编号唯一确定；岗位表中不冗余存放商家名称等商家属性，而通过商家编号外键关联获取，避免了由岗位编号到商家编号再到商家名称的传递依赖。因此，全部关系模式均满足第三范式（3NF）。

3．物理结构设计

各表结构和必要的索引等。根据逻辑结构设计结果，在 SQL Server 2019 中建立数据库 CampusPartTime，共包含 student、employer、admin、job、application、comment 六张数据表，各表的物理结构分别如表 2 至表 7 所示。

表 2 学生信息表（student）

字段名称	数据类型	主键	允许空	功能描述
student_id	int	是	否	学生编号，标识列自增
stu_no	varchar(20)	否	否	学号，唯一
stu_name	varchar(20)	否	否	学生姓名
stu_sex	nchar(1)	否	否	性别，取值男或女
stu_birth	date	否	是	出生日期
stu_college	varchar(50)	否	否	所在学院
stu_major	varchar(50)	否	否	所学专业
stu_grade	varchar(10)	否	否	年级，如 2023 级
stu_phone	varchar(11)	否	是	联系电话，11 位手机号
stu_email	varchar(50)	否	是	电子邮箱
stu_password	varchar(60)	否	否	密码，BCrypt 哈希值
reg_date	datetime	否	否	注册日期，默认取当前时间
stu_status	tinyint	否	否	状态，0 正常，1 禁用

表 3 商家信息表（employer）

字段名称	数据类型	主键	允许空	功能描述
employer_id	int	是	否	商家编号，标识列自增
emp_name	varchar(50)	否	否	商家名称
emp_contact	varchar(20)	否	否	联系人姓名
emp_phone	varchar(11)	否	否	联系电话，11 位手机号
emp_address	varchar(100)	否	是	商家地址
emp_intro	nvarchar(500)	否	是	商家简介
emp_email	varchar(50)	否	是	电子邮箱
emp_password	varchar(60)	否	否	密码，BCrypt 哈希值
reg_date	datetime	否	否	注册日期，默认取当前时间
emp_status	tinyint	否	否	状态，0 正常，1 禁用

表 4 管理员表（admin）

字段名称	数据类型	主键	允许空	功能描述
admin_id	int	是	否	管理员编号，标识列自增
admin_name	varchar(20)	否	否	管理员姓名
admin_password	varchar(60)	否	否	密码，BCrypt 哈希值

表 5 兼职岗位表（job）

字段名称	数据类型	主键	允许空	功能描述
job_id	int	是	否	岗位编号，标识列自增
employer_id	int	否	否	商家编号，外键
job_name	varchar(50)	否	否	岗位名称
job_type	varchar(20)	否	否	岗位类型，如家教、餐饮、促销
salary	decimal(8,2)	否	否	薪资数额
salary_unit	varchar(10)	否	否	计薪单位，元/小时或元/天
work_place	varchar(100)	否	否	工作地点
work_time	varchar(50)	否	否	工作时段
recruit_num	int	否	否	招聘人数，不小于 1
hired_num	int	否	否	已录用人数，默认 0
job_desc	nvarchar(1000)	否	是	岗位描述
publish_date	datetime	否	否	发布日期，默认取当前时间
deadline	date	否	否	报名截止日期
audit_status	tinyint	否	否	审核状态，0 待审核，1 通过，2 驳回
job_status	tinyint	否	否	岗位状态，0 在招，1 已招满，2 已下架

表 6 申请记录表（application）

字段名称	数据类型	主键	允许空	功能描述
app_id	int	是	否	申请编号，标识列自增
student_id	int	否	否	学生编号，外键
job_id	int	否	否	岗位编号，外键
app_date	datetime	否	否	申请日期，默认取当前时间
app_status	tinyint	否	否	申请状态，0 待处理，1 已录用，2 已拒绝
app_remark	nvarchar(200)	否	是	申请备注

表 7 评价表（comment）

字段名称	数据类型	主键	允许空	功能描述
comment_id	int	是	否	评价编号，标识列自增
student_id	int	否	否	学生编号，外键
job_id	int	否	否	岗位编号，外键
employer_id	int	否	否	商家编号，外键，标注被评价商家
score	int	否	否	评分，取值 1 至 5
content	nvarchar(500)	否	是	评价内容
comment_date	datetime	否	否	评价日期，默认取当前时间
audit_status	tinyint	否	否	审核状态，0 待审核，1 通过，2 驳回

各表的完整建表脚本如下，脚本中包含主键、外键、唯一约束、检查约束与默认值约束的定义。

```
CREATE DATABASE CampusPartTime;
USE CampusPartTime;
CREATE TABLE student (
    student_id INT IDENTITY(1,1) PRIMARY KEY, stu_no VARCHAR(20) NOT NULL UNIQUE,
    stu_name VARCHAR(20) NOT NULL, stu_sex NCHAR(1) NOT NULL
```

```
    CHECK (stu_sex IN (N'男', N'女')), stu_birth DATE NULL,
  stu_college VARCHAR(50) NOT NULL, stu_major VARCHAR(50) NOT NULL,
  stu_grade VARCHAR(10) NOT NULL, stu_phone VARCHAR(11) NULL,
  stu_email VARCHAR(50) NULL, stu_password VARCHAR(60) NOT NULL,
  reg_date DATETIME NOT NULL DEFAULT GETDATE(),
  stu_status TINYINT NOT NULL DEFAULT 0 CHECK (stu_status IN (0, 1)),
  CONSTRAINT CK_student_phone CHECK (stu_phone IS NULL OR stu_phone LIKE
    '1[0-9][0-9][0-9][0-9][0-9][0-9][0-9][0-9][0-9][0-9]')
);
CREATE UNIQUE INDEX UQ_student_phone ON student(stu_phone) WHERE stu_phone IS NOT NULL;
CREATE INDEX IX_student_reg_date ON student(reg_date);
CREATE TABLE employer (
  employer_id INT IDENTITY(1,1) PRIMARY KEY, emp_name VARCHAR(50) NOT NULL,
  emp_contact VARCHAR(20) NOT NULL, emp_phone VARCHAR(11) NOT NULL UNIQUE
    CHECK (emp_phone LIKE '1[0-9][0-9][0-9][0-9][0-9][0-9][0-9][0-9][0-9][0-9]'),
  emp_address VARCHAR(100) NULL, emp_intro NVARCHAR(500) NULL,
  emp_email VARCHAR(50) NULL, emp_password VARCHAR(60) NOT NULL,
  reg_date DATETIME NOT NULL DEFAULT GETDATE(),
  emp_status TINYINT NOT NULL DEFAULT 0 CHECK (emp_status IN (0, 1))
);
CREATE INDEX IX_employer_name ON employer(emp_name);
CREATE TABLE admin (
  admin_id INT IDENTITY(1,1) PRIMARY KEY,
  admin_name VARCHAR(20) NOT NULL UNIQUE, admin_password VARCHAR(60) NOT NULL
);
CREATE TABLE job (
  job_id INT IDENTITY(1,1) PRIMARY KEY,
  employer_id INT NOT NULL REFERENCES employer(employer_id),
  job_name VARCHAR(50) NOT NULL, job_type VARCHAR(20) NOT NULL,
  salary DECIMAL(8,2) NOT NULL DEFAULT 0 CHECK (salary >= 0),
  salary_unit VARCHAR(10) NOT NULL DEFAULT '元/小时'
    CHECK (salary_unit IN ('元/小时', '元/天')),
  work_place VARCHAR(100) NOT NULL, work_time VARCHAR(50) NOT NULL,
  recruit_num INT NOT NULL DEFAULT 1 CHECK (recruit_num BETWEEN 1 AND 100),
  hired_num INT NOT NULL DEFAULT 0, job_desc NVARCHAR(1000) NULL,
  publish_date DATETIME NOT NULL DEFAULT GETDATE(), deadline DATE NOT NULL,
  audit_status TINYINT NOT NULL DEFAULT 0 CHECK (audit_status IN (0, 1, 2)),
  job_status TINYINT NOT NULL DEFAULT 0 CHECK (job_status IN (0, 1, 2)),
  CONSTRAINT CK_job_hired CHECK (hired_num >= 0 AND hired_num <= recruit_num)
);
CREATE INDEX IX_job_employer_status ON job(employer_id, job_status);
CREATE INDEX IX_job_type ON job(job_type);
CREATE INDEX IX_job_publish_date ON job(publish_date DESC);
CREATE TABLE application (
  app_id INT IDENTITY(1,1) PRIMARY KEY,
  student_id INT NOT NULL REFERENCES student(student_id),
  job_id INT NOT NULL REFERENCES job(job_id),
  app_date DATETIME NOT NULL DEFAULT GETDATE(),
  app_status TINYINT NOT NULL DEFAULT 0 CHECK (app_status IN (0, 1, 2)),
  app_remark NVARCHAR(200) NULL,
  CONSTRAINT UQ_app_student_job UNIQUE (student_id, job_id)
);
CREATE INDEX IX_app_job_status ON application(job_id, app_status);
CREATE TABLE comment (
  comment_id INT IDENTITY(1,1) PRIMARY KEY,
  student_id INT NOT NULL REFERENCES student(student_id),
  job_id INT NOT NULL REFERENCES job(job_id),
  employer_id INT NOT NULL REFERENCES employer(employer_id),
  score INT NOT NULL DEFAULT 5 CHECK (score BETWEEN 1 AND 5),
  content NVARCHAR(500) NULL,
  comment_date DATETIME NOT NULL DEFAULT GETDATE(),
  audit_status TINYINT NOT NULL DEFAULT 0 CHECK (audit_status IN (0, 1, 2)),
  CONSTRAINT UQ_comment_stu_job UNIQUE (student_id, job_id)
);
CREATE INDEX IX_comment_employer ON comment(employer_id, audit_status);
```

数据库索引的设计说明如下：

（1）student 表在 stu_no 上通过唯一约束建立唯一索引，用于加速学号登录查询；stu_phone 上建立筛选唯一索引，保证同一手机号不被重复注册；reg_date 上建立普通索引，支持按注册时间统计。

（2）employer 表在 emp_name 上建立普通索引 IX_employer_name，加速商家名称的模糊检索；emp_phone 通过唯一约束建立唯一索引，保证联系电话不重复。

（3）job 表建立复合索引 IX_job_employer_status（employer_id, job_status），支持商家按状态查看本企业岗位；建立 IX_job_type 支持按岗位类型检索；建立 IX_job_publish_date（publish_date DESC）支持按发布时间倒序分页；建立筛选索引 IX_job_audit，仅索引待审核记录，加速管理员的待审核岗位查询。

（4）application 表通过 UNIQUE 约束建立复合唯一索引 UQ_app_student_job（student_id, job_id），从数据库层杜绝同一学生对同一岗位的重复申请；建立 IX_app_job_status（job_id, app_status）支持商家按岗位筛选待处理的申请记录。

（5）comment 表建立 IX_comment_job（job_id, audit_status）支持按岗位查询已审核评价；建立 IX_comment_employer 支持商家信誉统计；建立筛选索引 IX_comment_audit 加速待审核评价查询。

（6）六张表的主键均建立聚簇索引，由主键约束自动创建，用于支撑按主键的等值查询与关联连接。

4．数据字典

数据字典在物理结构的基础上补充长度、默认值与取值范围等约束信息，是后续编码实现与数据库脚本编写的统一依据。各表的数据字典分别如表 8 至表 13 所示。

表 8 学生实体数据字典（student）

字段名	数据类型	长度	允许空	默认值	取值范围	说明
student_id	int	4	否	自增	1～2147483647	学生编号
stu_no	varchar	20	否	无	8～20 位数字	学号，唯一
stu_name	varchar	20	否	无	1～20 个字符	学生姓名
stu_sex	nchar	1	否	无	男、女	性别
stu_birth	date	3	是	NULL	1900-01-01～当前	出生日期
stu_college	varchar	50	否	无	1～50 个字符	所在学院
stu_major	varchar	50	否	无	1～50 个字符	所学专业
stu_grade	varchar	10	否	无	如 2023 级	年级
stu_phone	varchar	11	是	NULL	11 位手机号	联系电话，唯一
stu_email	varchar	50	是	NULL	合法邮箱格式	电子邮箱
stu_password	varchar	60	否	无	60 位哈希串	BCrypt 哈希值
reg_date	datetime	8	否	GETDATE()	合法日期时间	注册日期
stu_status	tinyint	1	否	0	0、1	0 正常，1 禁用

表 9 商家实体数据字典（employer）

字段名	数据类型	长度	允许空	默认值	取值范围	说明
employer_id	int	4	否	自增	1～2147483647	商家编号
emp_name	varchar	50	否	无	1～50 个字符	商家名称
emp_contact	varchar	20	否	无	1～20 个字符	联系人
emp_phone	varchar	11	否	无	11 位手机号	联系电话，唯一
emp_address	varchar	100	是	NULL	1～100 个字符	商家地址
emp_intro	nvarchar	500	是	NULL	1～500 个字符	商家简介
emp_email	varchar	50	是	NULL	合法邮箱格式	电子邮箱
emp_password	varchar	60	否	无	60 位哈希串	BCrypt 哈希值

reg_date	datetime	8	否	GETDATE()	合法日期时间	注册日期
emp_status	tinyint	1	否	0	0、1	0 正常，1 禁用

表 10 管理员实体数据字典（admin）

字段名	数据类型	长度	允许空	默认值	取值范围	说明
admin_id	int	4	否	自增	1～2147483647	管理员编号
admin_name	varchar	20	否	无	1～20 个字符	管理员姓名，唯一
admin_password	varchar	60	否	无	60 位哈希串	BCrypt 哈希值

表 11 兼职岗位实体数据字典（job）

字段名	数据类型	长度	允许空	默认值	取值范围	说明
job_id	int	4	否	自增	1～2147483647	岗位编号
employer_id	int	4	否	无	已存在的商家编号	商家编号，外键
job_name	varchar	50	否	无	1～50 个字符	岗位名称
job_type	varchar	20	否	无	家教、餐饮、促销等	岗位类型
salary	decimal	8,2	否	0	0～999999.99	薪资数额
salary_unit	varchar	10	否	元/小时	元/小时、元/天	计薪单位
work_place	varchar	100	否	无	1～100 个字符	工作地点
work_time	varchar	50	否	无	1～50 个字符	工作时段
recruit_num	int	4	否	1	1～100	招聘人数
hired_num	int	4	否	0	0～recruit_num	已录用人数
job_desc	nvarchar	1000	是	NULL	1～1000 个字符	岗位描述
publish_date	datetime	8	否	GETDATE()	合法日期时间	发布日期
deadline	date	3	否	无	不早于发布日期	报名截止日期
audit_status	tinyint	1	否	0	0、1、2	0 待审核，1 通过，2 驳回
job_status	tinyint	1	否	0	0、1、2	0 在招，1 已招满，2 已下架

表 12 申请记录实体数据字典（application）

字段名	数据类型	长度	允许空	默认值	取值范围	说明
app_id	int	4	否	自增	1～2147483647	申请编号
student_id	int	4	否	无	已存在的学生编号	学生编号，外键
job_id	int	4	否	无	已存在的岗位编号	岗位编号，外键
app_date	datetime	8	否	GETDATE()	合法日期时间	申请日期
app_status	tinyint	1	否	0	0、1、2	0 待处理，1 已录用，2 已拒绝
app_remark	nvarchar	200	是	NULL	1～200 个字符	申请备注

表 13 评价实体数据字典（comment）

字段名	数据类型	长度	允许空	默认值	取值范围	说明
comment_id	int	4	否	自增	1～2147483647	评价编号
student_id	int	4	否	无	已存在的学生编号	学生编号，外键
job_id	int	4	否	无	已存在的岗位编号	岗位编号，外键
employer_id	int	4	否	无	已存在的商家编号	商家编号，外键
score	int	4	否	5	1～5	评分
content	nvarchar	500	是	NULL	1～500 个字符	评价内容
comment_date	datetime	8	否	GETDATE()	合法日期时间	评价日期
audit_status	tinyint	1	否	0	0、1、2	0 待审核，1 通过，2 驳回

表 8 至表 13 中的数据类型、长度、默认值与取值范围与表 2 至表 7 的物理结构设计完全一致，二者共同构成数据库实现的规范依据。

四．系统详细设计与实现

主要编码及描述。系统详细设计分为数据库服务器端与应用服务器端两部分：前者实现视图、存储过程与触发器，负责数据完整性约束与核心业务逻辑的自动化；后者基于 ASP.NET Web 窗体与 ADO.NET 实现页面交互与数据访问。

1．数据库服务器端

要求设计中至少使用视图、存储过程和触发器各 1 次。本系统共设计 1 个视图、3 个存储过程和 1 个触发器，下面给出所用视图、存储过程和触发器的定义及其作用。

（1）视图 v_job_detail 的定义如下。

```
CREATE VIEW v_job_detail
AS
SELECT j.job_id, j.job_name, j.job_type, j.salary, j.salary_unit, j.work_place,
    j.work_time, j.recruit_num, j.hired_num, j.job_desc, j.publish_date,
    j.deadline, j.job_status, e.employer_id, e.emp_name, e.emp_contact, e.emp_phone
FROM job AS j
INNER JOIN employer AS e ON j.employer_id = e.employer_id
WHERE j.audit_status = 1;
```

作用：将岗位表与商家表按商家编号连接，对外提供含商家名称与联系方式的完整岗位信息，并在视图层过滤未通过审核的岗位，应用层无需重复编写多表连接语句，既降低耦合度，也避免未审核信息泄露。

（2）存储过程 proc_apply_job 的定义如下。

```
CREATE PROCEDURE proc_apply_job @student_id INT, @job_id INT, @result INT OUTPUT
AS
BEGIN
  SET NOCOUNT ON; SET @result = -4;
  DECLARE @v_audit TINYINT, @v_status TINYINT, @v_deadline DATE;
  SELECT @v_audit = j.audit_status, @v_status = j.job_status, @v_deadline = j.deadline
  FROM job AS j WHERE j.job_id = @job_id;
  IF @@ROWCOUNT = 0 OR @v_audit <> 1 BEGIN SET @result = -1; RETURN; END
  IF @v_status <> 0 OR @v_deadline < CAST(GETDATE() AS DATE) BEGIN SET @result = -2; RETURN; END
  IF EXISTS (SELECT 1 FROM application AS a
        WHERE a.student_id = @student_id AND a.job_id = @job_id)
  BEGIN SET @result = -3; RETURN; END
  BEGIN TRY
    BEGIN TRANSACTION;
    INSERT INTO application (student_id, job_id, app_date, app_status)
    VALUES (@student_id, @job_id, GETDATE(), 0);
    COMMIT TRANSACTION; SET @result = 1;
```

```
    END TRY
    BEGIN CATCH
      IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION;
      SET @result = -4;
    END CATCH
  END
```

作用：将岗位审核状态校验、报名截止校验、重复申请校验与申请记录写入集中到数据库端一次完成，并使用显式事务保证原子性。该过程只写入状态为待处理的申请记录，不修改已录用人数。返回码：1 成功，-1 岗位不存在或未通过审核，-2 岗位不在招或已过报名截止日期，-3 已申请过，-4 写入失败。

（3）存储过程 proc_hire_applicant 的定义如下。

```
CREATE PROCEDURE proc_hire_applicant @app_id INT, @employer_id INT, @result INT OUTPUT
AS
BEGIN
  SET NOCOUNT ON; SET @result = -4;
  DECLARE @v_job_id INT, @v_app_status TINYINT, @v_recruit INT,
      @v_hired INT, @v_job_status TINYINT;
  BEGIN TRY
    BEGIN TRANSACTION;
    SELECT @v_job_id = a.job_id, @v_app_status = a.app_status
    FROM application AS a INNER JOIN job AS j ON a.job_id = j.job_id
    WHERE a.app_id = @app_id AND j.employer_id = @employer_id;
    IF @@ROWCOUNT = 0 BEGIN ROLLBACK; SET @result = -1; RETURN; END
    IF @v_app_status <> 0 BEGIN ROLLBACK; SET @result = -2; RETURN; END
    SELECT @v_recruit = j.recruit_num, @v_hired = j.hired_num,
        @v_job_status = j.job_status
    FROM job AS j WITH (UPDLOCK, ROWLOCK) WHERE j.job_id = @v_job_id;
    IF @v_job_status <> 0 OR @v_hired >= @v_recruit
    BEGIN ROLLBACK; SET @result = -3; RETURN; END
    UPDATE application SET app_status = 1 WHERE app_id = @app_id;
    UPDATE job SET hired_num = hired_num + 1 WHERE job_id = @v_job_id;
    COMMIT TRANSACTION; SET @result = 1;
  END TRY
  BEGIN CATCH
    IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION;
    SET @result = -4;
  END CATCH
END
```

作用：完成录用并保证并发一致性。过程先校验申请属于当前商家的岗位且处于待处理状态，再在事务内以 UPDLOCK 提示重新读取招聘人数与已录用人数，仅在未满员时把申请状态置为已录用、已录用人数加一。返回码：1 成功，-1 申请不存在或不属于本商家，-2 该申请已处理，-3 岗位已招满或不在招，-4 执行异常。

（4）存储过程 proc_reject_applicant 的定义如下。

```
CREATE PROCEDURE proc_reject_applicant
  @app_id INT, @employer_id INT, @result INT OUTPUT
AS
BEGIN
  SET NOCOUNT ON;
  UPDATE a SET a.app_status = 2
  FROM application AS a INNER JOIN job AS j ON a.job_id = j.job_id
  WHERE a.app_id = @app_id AND j.employer_id = @employer_id AND a.app_status = 0;

  IF @@ROWCOUNT = 1 SET @result = 1; ELSE SET @result = -1;
END
```

作用：校验申请归属后把待处理申请置为已拒绝，不改变已录用人数；以一条带条件的 UPDATE 同时完成校验与更新，避免“先查后改”的并发窗口。

（5）触发器 trg_job_after_apply 的定义如下。

```
CREATE TRIGGER trg_job_after_apply ON application AFTER UPDATE
AS
BEGIN
  SET NOCOUNT ON;
  IF NOT UPDATE(app_status) RETURN;
  UPDATE j SET j.job_status = 1
  FROM job AS j INNER JOIN (
    SELECT DISTINCT i.job_id FROM inserted AS i
    INNER JOIN deleted AS d ON i.app_id = d.app_id
```

```
        WHERE i.app_status = 1 AND d.app_status <> 1) AS t ON j.job_id = t.job_id
    WHERE j.job_status = 0 AND j.hired_num >= j.recruit_num;
END
```

作用：在申请状态更新后自动维护岗位状态。触发器采用 AFTER UPDATE 集合式写法，通过 inserted 与 deleted 表一次性取出本次由其他状态变为已录用的岗位编号并去重，再以一条 UPDATE 批量修正状态，批量录用时不会遗漏行；已下架岗位的状态不被覆盖。

2．应用服务器端

应用服务器端统一通过 ConfigurationManager.ConnectionStrings["CampusConn"] 获取连接字符串，数据访问均以 using 语句管理连接并使用 SqlParameter 参数化。下面按录入、删除、检索、修改四类功能各举一例，并补充录用与拒绝、评价、审核、统计及权限拦截实现，最后给出关键前端代码。

（1）数据录入——商家发布岗位（JobPublish.aspx）。

```
protected void btnPublish_Click(object sender, EventArgs e)
{
    int employerId = Convert.ToInt32(Session["EmployerId"]);
    string sql = @"INSERT INTO job (employer_id, job_name, job_type, salary, salary_unit,
        work_place, work_time, recruit_num, job_desc, publish_date, deadline,
        audit_status, job_status) VALUES (@employer_id, @job_name, @job_type, @salary,
        @salary_unit, @work_place, @work_time, @recruit_num, @job_desc, GETDATE(),
        @deadline, 0, 0)";
    using (SqlConnection conn = new SqlConnection(cs))
    using (SqlCommand cmd = new SqlCommand(sql, conn))
    {
        cmd.Parameters.AddWithValue("@employer_id", employerId);
        cmd.Parameters.AddWithValue("@job_name", txtJobName.Text.Trim());
        cmd.Parameters.AddWithValue("@job_type", ddlJobType.SelectedValue);
        cmd.Parameters.AddWithValue("@salary", decimal.Parse(txtSalary.Text));
        cmd.Parameters.AddWithValue("@salary_unit", ddlUnit.SelectedValue);
        cmd.Parameters.AddWithValue("@work_place", txtPlace.Text.Trim());
        cmd.Parameters.AddWithValue("@work_time", txtTime.Text.Trim());
        cmd.Parameters.AddWithValue("@recruit_num", int.Parse(txtNum.Text));
        cmd.Parameters.AddWithValue("@job_desc", txtDesc.Text.Trim());
        cmd.Parameters.AddWithValue("@deadline", Convert.ToDateTime(txtDeadline.Text));
        conn.Open(); cmd.ExecuteNonQuery();
    }
    lblMsg.Text = "岗位发布成功，等待管理员审核！"; BindJobList();
}
```

说明：employer_id 取自 Session，避免客户端篡改；新记录审核状态与岗位状态均取 0，管理员审核通过后方在前台展示。

（2）数据删除——商家下架岗位（JobManage.aspx）。

```
protected void btnDelete_Click(object sender, EventArgs e)
{
    Button btn = (Button)sender;
    int jobId = Convert.ToInt32(btn.CommandArgument);
    int employerId = Convert.ToInt32(Session["EmployerId"]);
    string sql = @"UPDATE job SET job_status = 2
                WHERE job_id = @job_id AND employer_id = @employer_id";
    using (SqlConnection conn = new SqlConnection(cs))
    using (SqlCommand cmd = new SqlCommand(sql, conn))
    {
        cmd.Parameters.AddWithValue("@job_id", jobId);
        cmd.Parameters.AddWithValue("@employer_id", employerId);
        conn.Open();
        int n = cmd.ExecuteNonQuery();
        lblMsg.Text = n > 0 ? "岗位已下架！" : "操作失败，无权操作该岗位！";
    }
    BindJobList();
}
```

说明：采用状态更新的逻辑删除，保留历史申请与评价数据；WHERE 同时限定 employer_id，防止篡改 URL 越权操作他人岗位。

（3）数据检索——学生组合条件分页检索（JobList.aspx）。

```
protected void btnSearch_Click(object sender, EventArgs e)
{
```

```
    int pageIndex = ViewState["PageIndex"] == null ? 1 : (int)ViewState["PageIndex"];
    const int pageSize = 10;
    StringBuilder sb = new StringBuilder(
       @"SELECT job_id, job_name, job_type, salary, salary_unit, work_place,
              work_time, emp_name, publish_date FROM v_job_detail WHERE 1 = 1");
    List<SqlParameter> ps = new List<SqlParameter>();
    if (ddlJobType.SelectedIndex > 0)
    { sb.Append(" AND job_type = @job_type");
      ps.Add(new SqlParameter("@job_type", ddlJobType.SelectedValue)); }
    if (!string.IsNullOrEmpty(txtSalary.Text))
    { sb.Append(" AND salary >= @salary");
      ps.Add(new SqlParameter("@salary", decimal.Parse(txtSalary.Text))); }
    if (!string.IsNullOrEmpty(txtKeyword.Text.Trim()))
    { sb.Append(" AND job_name LIKE @kw");
      ps.Add(new SqlParameter("@kw", "%" + txtKeyword.Text.Trim() + "%")); }
    sb.Append(@" ORDER BY publish_date DESC
              OFFSET @offset ROWS FETCH NEXT @pageSize ROWS ONLY");
    ps.Add(new SqlParameter("@offset", (pageIndex - 1) * pageSize));
    ps.Add(new SqlParameter("@pageSize", pageSize));
    using (SqlConnection conn = new SqlConnection(cs))
    using (SqlCommand cmd = new SqlCommand(sb.ToString(), conn))
    using (SqlDataAdapter da = new SqlDataAdapter(cmd))
    {
       cmd.Parameters.AddRange(ps.ToArray());
       DataTable dt = new DataTable(); da.Fill(dt);
       gvJobList.DataSource = dt; gvJobList.DataBind();
       lblPage.Text = string.Format("第 {0} 页", pageIndex);
    }
 }
```

说明：检索条件全部参数化，杜绝 SQL 注入；基于视图查询自动排除未审核岗位；OFFSET-FETCH 只返回当前页数据，按发布日期倒序展示。

（4）数据修改——商家修改岗位（JobEdit.aspx）。

```
 protected void btnUpdate_Click(object sender, EventArgs e)
 {
    int jobId = Convert.ToInt32(Request.QueryString["job_id"]);
    int employerId = Convert.ToInt32(Session["EmployerId"]);
    string sql = @"UPDATE job SET job_name = @job_name, job_type = @job_type,
       salary = @salary, salary_unit = @salary_unit, work_place = @work_place,
       work_time = @work_time, recruit_num = @recruit_num, job_desc = @job_desc,
       deadline = @deadline, audit_status = 0
       WHERE job_id = @job_id AND employer_id = @employer_id AND hired_num <= @recruit_num";
    using (SqlConnection conn = new SqlConnection(cs))
    using (SqlCommand cmd = new SqlCommand(sql, conn))
    {
       cmd.Parameters.AddWithValue("@job_name", txtJobName.Text.Trim());
       cmd.Parameters.AddWithValue("@job_type", ddlJobType.SelectedValue);
       cmd.Parameters.AddWithValue("@salary", decimal.Parse(txtSalary.Text));
       cmd.Parameters.AddWithValue("@salary_unit", ddlUnit.SelectedValue);
       cmd.Parameters.AddWithValue("@work_place", txtPlace.Text.Trim());
       cmd.Parameters.AddWithValue("@work_time", txtTime.Text.Trim());
       cmd.Parameters.AddWithValue("@recruit_num", int.Parse(txtNum.Text));
       cmd.Parameters.AddWithValue("@job_desc", txtDesc.Text.Trim());
       cmd.Parameters.AddWithValue("@deadline", Convert.ToDateTime(txtDeadline.Text));
       cmd.Parameters.AddWithValue("@job_id", jobId);
       cmd.Parameters.AddWithValue("@employer_id", employerId);
       conn.Open();
       lblMsg.Text = cmd.ExecuteNonQuery() > 0 ? "岗位修改成功，等待管理员重新审核！"
                                               : "修改失败，招聘人数不得少于已录用人数！";
    }
 }
```

说明：修改后审核状态重置为 0，需重新审核，防止商家改内容绕过审核；hired_num 不超过 recruit_num 的条件保证招聘人数不被改小于已录用人数。

（5）录用与拒绝——商家处理申请（ApplicationManage.aspx）。

```
 protected void gvApply_RowCommand(object sender, GridViewCommandEventArgs e)
 {
    if (e.CommandName != "Hire" && e.CommandName != "Reject") return;
    int appId = Convert.ToInt32(e.CommandArgument);
    int employerId = Convert.ToInt32(Session["EmployerId"]);
    string proc = e.CommandName == "Hire" ? "proc_hire_applicant" : "proc_reject_applicant";
    int result;
    using (SqlConnection conn = new SqlConnection(cs))
    using (SqlCommand cmd = new SqlCommand(proc, conn))
    {
```

```
        cmd.CommandType = CommandType.StoredProcedure;
        cmd.Parameters.AddWithValue("@app_id", appId);
        cmd.Parameters.AddWithValue("@employer_id", employerId);
        SqlParameter p = cmd.Parameters.Add("@result", SqlDbType.Int);
        p.Direction = ParameterDirection.Output;
        conn.Open(); cmd.ExecuteNonQuery();
        result = Convert.ToInt32(p.Value);
    }
    switch (result)
    {
        case 1: lblMsg.Text = e.CommandName == "Hire" ? "录用成功！" : "已拒绝该申请！"; break;
        case -1: lblMsg.Text = "操作失败，该申请不属于您的岗位！"; break;
        case -2: lblMsg.Text = "该申请已处理，请勿重复操作！"; break;
        case -3: lblMsg.Text = "岗位已招满，无法继续录用！"; break;
        default: lblMsg.Text = "操作异常，请稍后重试！"; break;
    }
    BindApplyList();
}
```

说明：业务校验与状态变更在数据库端一次执行，应用层只按返回码提示；录用时同步增加已录用人数，满员后触发器自动置为已招满。

（6）评价提交——学生提交评价（JobDetail.aspx）。

```
protected void btnComment_Click(object sender, EventArgs e)
{
    int studentId = Convert.ToInt32(Session["StudentId"]);
    int jobId = Convert.ToInt32(Request.QueryString["job_id"]);
    using (SqlConnection conn = new SqlConnection(cs))
    {
        conn.Open();
        string chk = @"SELECT COUNT(1) FROM application
                       WHERE student_id = @sid AND job_id = @jid AND app_status = 1";
        using (SqlCommand c1 = new SqlCommand(chk, conn))
        {
            c1.Parameters.AddWithValue("@sid", studentId);
            c1.Parameters.AddWithValue("@jid", jobId);
            if (Convert.ToInt32(c1.ExecuteScalar()) == 0)
            { lblMsg.Text = "仅已录用的兼职才可以评价！"; return; }
        }
        string sql = @"INSERT INTO comment (student_id, job_id, employer_id, score,
                         content, comment_date, audit_status)
                       SELECT @sid, @jid, j.employer_id, @score, @content, GETDATE(), 0
                       FROM job AS j WHERE j.job_id = @jid";
        using (SqlCommand c2 = new SqlCommand(sql, conn))
        {
            c2.Parameters.AddWithValue("@sid", studentId);
            c2.Parameters.AddWithValue("@jid", jobId);
            c2.Parameters.AddWithValue("@score", Convert.ToInt32(rblScore.SelectedValue));
            c2.Parameters.AddWithValue("@content", txtComment.Text.Trim());
            c2.ExecuteNonQuery();
        }
    }
    lblMsg.Text = "评价提交成功，管理员审核后展示！";
}
```

说明：先校验申请状态为已录用方可评价；employer_id 由岗位关联取得，使评价同属岗位与商家；写入时审核状态置 0；展示前统一调用 HttpUtility.HtmlEncode 对评价内容与用户输入做输出编码，把尖括号等字符转义为实体，防止留言式脚本注入。

（7）管理员审核——岗位审核与评价审核（AdminManage.aspx）。

```
protected void btnAuditJob_Click(object sender, EventArgs e)
{
    string[] args = ((Button)sender).CommandArgument.Split('|');
    string sql = @"UPDATE job SET audit_status = @audit,
        job_status = CASE WHEN @audit = 1 THEN 0 ELSE 2 END WHERE job_id = @job_id";
    using (SqlConnection conn = new SqlConnection(cs))
    using (SqlCommand cmd = new SqlCommand(sql, conn))
    {
        cmd.Parameters.AddWithValue("@audit", Convert.ToByte(args[1]));
        cmd.Parameters.AddWithValue("@job_id", Convert.ToInt32(args[0]));
        conn.Open(); cmd.ExecuteNonQuery();
    }
    lblMsg.Text = args[1] == "1" ? "岗位审核通过！" : "岗位已驳回！"; BindAuditList();
}
```

```
protected void btnAuditComment_Click(object sender, EventArgs e)
{
    string[] args = ((Button)sender).CommandArgument.Split('|');
    using (SqlConnection conn = new SqlConnection(cs))
    using (SqlCommand cmd = new SqlCommand(
        "UPDATE comment SET audit_status = @audit WHERE comment_id = @cid", conn))
    {
        cmd.Parameters.AddWithValue("@audit", Convert.ToByte(args[0]));
        cmd.Parameters.AddWithValue("@cid", Convert.ToInt32(args[1]));
        conn.Open(); cmd.ExecuteNonQuery();
    }
    lblMsg.Text = "评价审核完成！"; BindCommentList();
}
```

说明：审核通过时岗位置为在招、驳回时置为已下架，一次更新同步完成；审核入口仅对管理员角色开放。

（8）数据统计——平台运行指标查询（AdminManage.aspx）。

```
private DataTable GetStatistics()
{
    string sql = @"SELECT
        (SELECT COUNT(1) FROM student WHERE stu_status = 0) AS 注册学生数,
        (SELECT COUNT(1) FROM employer WHERE emp_status = 0) AS 注册商家数,
        (SELECT COUNT(1) FROM job WHERE audit_status = 1 AND job_status = 0) AS 在招岗位数,
        (SELECT COUNT(1) FROM application) AS 申请记录数,
        (SELECT COUNT(1) FROM application WHERE app_status = 1) AS 已录用数,
        (SELECT COUNT(1) FROM comment WHERE audit_status = 0) AS 待审核评价数";
    using (SqlConnection conn = new SqlConnection(cs))
    using (SqlCommand cmd = new SqlCommand(sql, conn))
    using (SqlDataAdapter da = new SqlDataAdapter(cmd))
    {
        DataTable dt = new DataTable(); da.Fill(dt); return dt;
    }
}
```

说明：以标量子查询一次返回六项指标，减少往返；在招岗位数同时限定审核通过与状态在招，结果以卡片展示。

（9）页面级权限拦截——页面基类 PageBase。

```
public class PageBase : System.Web.UI.Page
{
    protected virtual string[] AllowRoles { get { return new string[] { }; } }
    protected override void OnInit(EventArgs e)
    {
        base.OnInit(e);
        if (Session["RoleId"] == null) { Response.Redirect("~/Login.aspx", true); return; }
        if (AllowRoles.Length > 0 && Array.IndexOf(AllowRoles, Session["RoleId"].ToString()) < 0)
            Response.Redirect("~/Login.aspx", true);
    }
}
// 后台管理页仅允许管理员访问
public partial class AdminManage : PageBase
{
    protected override string[] AllowRoles { get { return new string[] { "admin" }; } }
}
```

说明：业务页面继承 PageBase，在 OnInit 阶段统一校验 Session 角色标识，未登录与越权角色直接重定向登录页，实现基于角色的访问控制，拦截先于页面事件执行，无法通过构造 URL 绕过。

（10）关键 HTML、CSS 与 JavaScript 代码（Login.aspx）。

```
<h2>校园兼职招聘管理系统</h2>
<div class="login-box">
  <asp:TextBox ID="txtUser" runat="server" CssClass="input-text" placeholder="用户名" />
  <asp:TextBox ID="txtPwd" runat="server" CssClass="input-text"
          TextMode="Password" placeholder="密码" />
  <asp:DropDownList ID="ddlRole" runat="server" CssClass="input-text">
    <asp:ListItem Value="student">学生</asp:ListItem>
    <asp:ListItem Value="employer">商家</asp:ListItem>
    <asp:ListItem Value="admin">管理员</asp:ListItem>
  </asp:DropDownList>
  <asp:Button ID="btnLogin" runat="server" Text="登 录" CssClass="btn-primary"
          OnClick="btnLogin_Click" OnClientClick="return checkLoginForm();" />
  <asp:Label ID="lblMsg" runat="server" CssClass="msg-error" />
</div>
```

```
<style>
body { font-family: "微软雅黑", Arial, sans-serif; background: #F5F7FA; font-size: 14px; }
.login-box { width: 400px; margin: 100px auto; background: #fff; padding: 30px;
         border-radius: 8px; box-shadow: 0 2px 10px rgba(0,0,0,.1); }
.login-box h2 { text-align: center; color: #2C6CB0; font-size: 18px; margin-bottom: 20px; }
.input-text { width: 100%; height: 36px; border: 1px solid #ccc; border-radius: 4px;
          padding: 0 8px; margin-bottom: 12px; }
.btn-primary { width: 100%; height: 40px; background: #2C6CB0; color: #fff;
           border: none; border-radius: 4px; cursor: pointer; }
.btn-primary:hover { background: #1E4F85; }
.msg-error { display: block; color: #E74C3C; font-size: 12px; margin-top: 8px; }
</style>
<script type="text/javascript">
function checkLoginForm() {
   var user = document.getElementById("<%=txtUser.ClientID%>");
   var pwd  = document.getElementById("<%=txtPwd.ClientID%>");
   var msg  = document.getElementById("<%=lblMsg.ClientID%>");
   if (user.value.trim() === "") { msg.innerHTML = "请输入用户名！"; user.focus(); return false; }
   if (pwd.value.length === 0) { msg.innerHTML = "请输入密码！"; pwd.focus(); return false; }
   if (pwd.value.length < 6) { msg.innerHTML = "密码长度不能少于 6 位！"; pwd.focus(); return false; }
   return true;
}
</script>
```

说明：页面为卡片式居中布局，统一引用 style.css 规范；前端校验减少无效请求，后台按角色分别查询三张用户表并用 BCrypt.Verify 比对密码，通过后写入 Session 并跳转对应首页。

3．设计结果主要界面展示及简要说明

系统共设计 9 个主要网页，满足网页数量不少于 5 个的要求；页面控件涵盖 TextBox 、 DropDownList 、 RadioButton 、 CheckBox 、 Button 、 Label 、 GridView、Repeater、Calendar、FileUpload 等 10 种以上。各页面布局、控件与业务流说明如下。

（1）Login.aspx 登录页。卡片式居中布局，含用户名框、密码框、角色下拉框与登录按钮；前端校验通过后提交，后台按角色查表并比对 BCrypt 哈希，成功后写入 Session 跳转对应首页，失败时红字提示原因。布局如图 3 所示。



图 3 登录页面布局示意图

（2）StudentRegister.aspx 学生注册页。纵向表单含学号、姓名、性别单选、学院、专业、年级、联系电话、邮箱与密码，前端校验手机号格式与密码一致性，后台校验学号唯一并哈希存密码，注册后待管理员审核。

（3）EmployerRegister.aspx 商家注册页。含商家名称、联系人、联系电话、地址、商家简介与密码，简介为多行文本框且限 500 字，提交后写入商家表，审核通过方可登录发布岗位。

（4）JobList.aspx 岗位检索页。上部为置顶导航栏，中部为岗位类型、最低薪资与关键词查询区，下部为 GridView 结果列表并配分页控件；点击查看进入详情页。布局如图 4 所示。

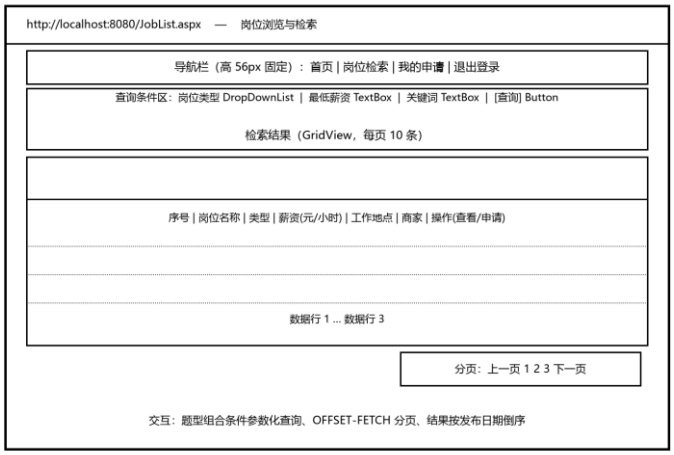


图 4 岗位检索页面布局示意图

（5）JobDetail.aspx 岗位详情页。展示岗位名称、类型、薪资及计薪单位、地点、时段、招聘与已录用人数、报名截止日期、描述及商家联系方式；申请按钮按申请状态动态显示为申请、待处理、已录用或已拒绝，已录用学生在下方以 1～5 星评分与文字框提交评价。

（6）JobPublish.aspx 岗位发布页。含岗位名称、类型、薪资与计薪单位、地点、时段、招聘人数、报名截止日期（Calendar 控件）与描述，宣传图通过 FileUpload 上传，服务端先校验扩展名白名单（.jpg、.png）与文件大小（不超过 2MB），再以 GUID 重命名保存至指定目录，避免可执行文件上传；提交后置为待审核。

（7）ApplicationManage.aspx 申请管理页。左侧为岗位下拉列表，右侧为申请 GridView，列出学生姓名、学院、联系电话、申请日期与状态，操作列提供录用与拒绝按钮，按存储过程返回码提示并刷新列表。

（8）MyApplication.aspx 我的申请页。以状态标签页分类展示待处理、已录用与已拒绝记录，已录用记录提供去评价入口，形成申请到评价的闭环。

（9）AdminManage.aspx 管理员后台。选项卡布局，含用户审核、岗位审核、评价审核与数据统计四个页签，分别提供账号启停、岗位审核、评价审核与运行指标卡片展示。

五．系统说明

5.1 系统开发环境

系统开发与运行所需的硬件与软件环境如下。

（1）服务器硬件配置要求如表 14 所示。

表 14 服务器硬件配置表（server_config）

配置项	最低要求	推荐配置	用途说明
CPU	双核 2.0GHz	四核 3.0GHz 及以上	承载 Web 服务与数据库服务
内存	4GB	8GB 及以上	支撑 IIS 工作进程与 SQL Server 缓冲池
磁盘	80GB HDD	120GB SSD 及以上	存放系统、应用程序与数据库文件

网络	10Mbps 共享	100Mbps 独享	满足页面传输与并发访问需求

（2）服务器软件：Windows Server 2019（64 位）或 Windows 10/11（64 位）专业版；IIS 10.0 并启用 ASP.NET 运行环境；SQL Server 2019 及 SQL Server Management Studio 18；.NET Framework 4.7.2 及以上。

（3）开发工具：Visual Studio 2019（“ASP.NET 和 Web 开发”工作负载）；前端使用 HTML5、CSS3、JavaScript 与 ASP.NET 服务器控件；数据访问使用 ADO.NET；版本管理使用 Git（可选）。

（4）客户端环境：Chrome 90 及以上、Edge 90 及以上或 Firefox 88 及以上浏览器；分辨率 1366×768 及以上；Windows 7 SP1 及以上、主流 Linux 或 macOS 10.13 及以上。

5.2 系统安装、配置与发布应用程序的步骤

（1）安装 SQL Server 2019 与 Visual Studio 2019，通过“服务器管理器”添加 IIS 角色，并启用 ASP.NET 与 ISAPI 扩展功能。

（2）在 SSMS 中执行脚本 CampusPartTime.sql，创建数据库 CampusPartTime、六张数据表及其主键、外键、UNIQUE、CHECK 与 DEFAULT 约束、各类索引、视图、存储过程与触发器，并插入管理员初始账号。

（3）创建最小权限数据库账号 cp_app_user，仅授予其 CampusPartTime 的 db_datareader 与 db_datawriter 角色，并额外授予三个存储过程的 EXECUTE 权限，不授予 sysadmin、db_owner 等高级权限，把可能的注入风险限制在业务数据范围内。

（4）在 Web.config 的 connectionStrings 节点中配置名称为 CampusConn 的连接，使用该独立账号及 16 位以上强口令；部署完成后执行 aspnet_regiis -pef "connectionStrings" "站点物理路径"加密配置节，运行时由 IIS 自动解密，配置文件中呈现密文；迁移站点前用-pdf 解密，迁移后重新加密。

（5）启用 Forms 认证，把 Login.aspx 设为登录页并拒绝匿名用户访问业务目录，同时在 PageBase 的 OnInit 中校验 Session 角色标识，二者共同构成页面级访问控制。

（6）在 Visual Studio 中发布项目到本地文件夹，再在 IIS 中新建网站指向该文件夹，应用程序池的.NET CLR 版本选择 v4.0、托管管道模式选择“集成”。

（7）访问 http://localhost:8080/Login.aspx，用预置管理员账号登录配置基础数据，再注册学生与商家账号，经审核后即可验证岗位发布、检索、申请、录用与评价的完整流程。

六．课程设计总结

本次课程设计按照软件工程方法完整经历了需求分析、总体设计、数据库设计与详细设计四个阶段，开发了基于 B/S 架构的校园兼职招聘管理系统。系统以 ASP.NET（C#）与 SQL Server 2019 为实现平台，采用三层架构组织代码，功能覆盖学生注册登录、商家岗位发布与维护、管理员内容审核、学生在线申请、商家录用决策与双向评价，最终交付 6 张数据表、1 个视图、3 个存储过程、1 个触发器与 9 个主要网页。

设计过程中解决了三个关键难点：其一，最初把已录用人数累加放在申请环节，导致申请即被视为录用，改进后把人数变更迁移到录用环节，新增 proc_hire_applicant 在事务中结合更新锁完成满员判断与人数累加；其二，逐行

读取 inserted 表的触发器在批量更新时会漏行，改为 AFTER UPDATE 集合式写法；其三，通过视图 v_job_detail 整合岗位与商家信息，简化查询并在视图层过滤未审核数据。

系统仍存在不足：一是未适配移动端，后续可采用响应式布局或开发微信小程序；二是岗位真实性依赖人工审核，缺乏商家资质认证与信用评价机制；三是缺少消息通知，申请状态变化需用户主动刷新，后续可引入邮件或短信通知。

通过本次设计，我掌握了从需求抽象到数据库落地的完整流程，实践了 E-R 图设计、关系模式转换与第三范式判定，熟悉了 SQL Server 中视图、存储过程与触发器的编写方法，也体会到参数化查询、权限拦截与配置加密等安全措施的必要性。

七．规范性要求

（1）格式参照《系统设计文档参考模板》，正文页数控制在 18～22 页，字体为宋体与 Times New Roman，正文为小四号字，采用单倍行距。

（2）每人一个文件夹，文件夹以“学号姓名”命名，例如“241240048 黄潇萱”。

（3）文件夹内应包含四项内容：①系统设计报告（Word 版含电子签，另附 PDF 版）；②程序源代码；③数据库文件或数据库备份；④运行环境及运行说明。

（4）全部材料压缩后上传至学习通考试模块提交。